

VirgilNN: Vision-based Image Recognition Geocoder for Identifying Locations using a Neural Network

Osnovi računarske inteligencije - predlog projekta

Momir Stanišić SV39/2022

Luka Bursać SV22/2022



1. Definicija problema

Problem geoenkodovanja predstavlja problem određivanja GPS koordinata na osnovu određene informacije. U ovom slučaju GPS koordinate se pronalaze na osnovu datih fotografija. Za određivanje jedne lokacije će se koristiti više fotografija različitih orijentacija. Problem će biti ograničen na traženje lokacija unutar Evrope.

2. Motivacija

Geoenkodovanje na osnovu slike može imati značajnu primenu u spasilačkim i drugim državnim službama. Ako je potrebno pronaći osobu, a postoji trag o njenoj poslednjoj lokaciji u vidu slike, ovakav model bi mogao odrediti gde se osoba nalazi. Model bi taokde mogao pronaći primenu u automatskom tagovanju lokacije slika, za potrebe dalje obrade putem algoritama kao što su feed algoritmi društvenih mreža.

3. Skup podataka

Za potrebe ovog projekta nije pronađen odgovarajući skup podataka, umesto toga kreiran je novi korišćenjem [Google Street View Static API](#) kao izvora. Izabrane su samo zemlje unutar Evrope koje imaju zadovoljavajuću pokrivenost. Problem geoenkodovanja ćemo predstaviti delom kao problem klasifikacije. U kontekstu ovog projekta, jedna klasa je predstavljena kao jedna ćelija na mapi. Prilikom generisanja ćelija u razmatranje je uzeta zakrivljenost planete Zemlje. Dužina ivice jedne ćelije iznosi 180km, sa tim što su ćelije manje od 9000km² spojene sa susednim ćelijama. Parametri su dobijeni empirijskom metodom.

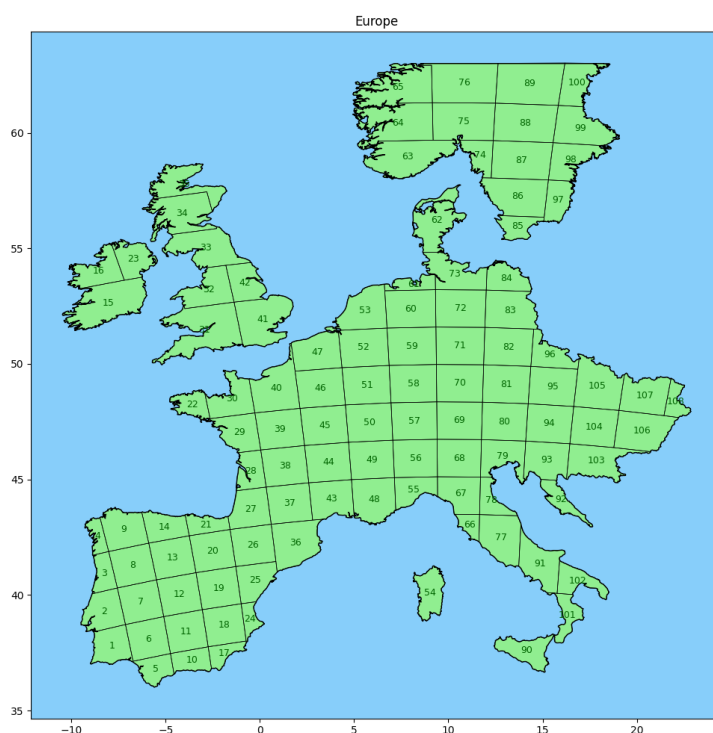


Figura 1: Mapa odabranih zemalja Evrope sa generisanim ćelijama

Ovom metodom je dobijeno 108 ćelija i za svaku od njih je nasumično odabrano 300 lokacija. Zasad lakšeg prikupljanja podataka i kasnijeg treniranja modela, ovaj skup lokacija je izdijeljen na tri ekvivalentna dela, svaki sa po 100 lokacija po ćeliji.

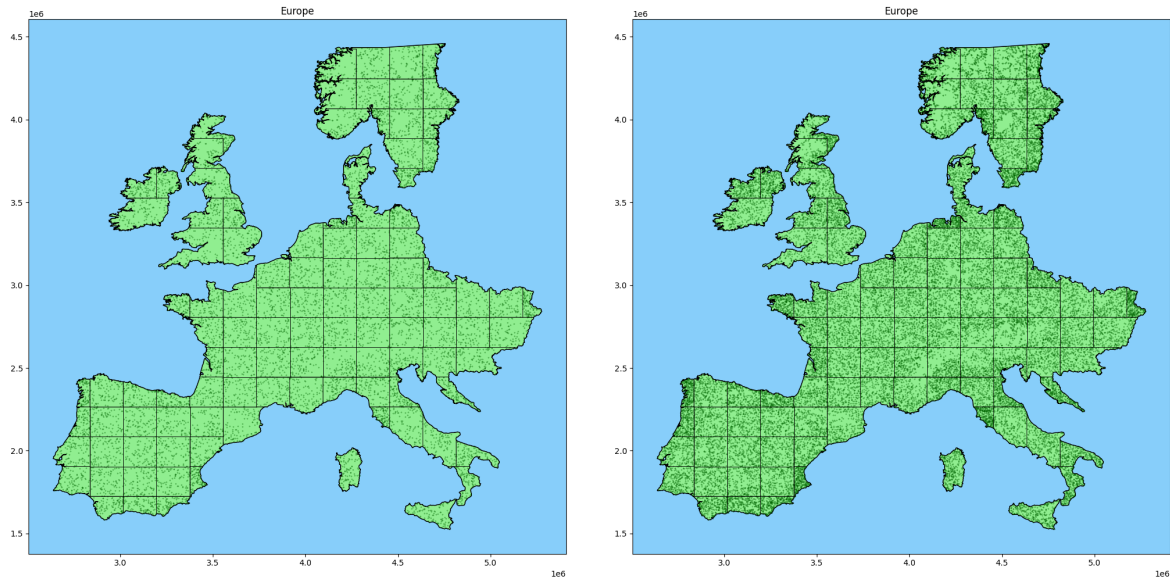


Figura 2: Vizuelizacija skupa podataka sa 100 i 300 slika po ćeliji

Za svaku lokaciju je uzeto po 5 slika, iz 5 različitih uglova. Jedna slika pokriva 72° vidnog polja i njena rezolucija je 640×460 .



Figura 3: Pet slika jedne lokacije koje predstavljaju pet različitih orijentacija

Na [ovoj adresi](#) moguće je naći skupove podataka sa 100, 200 i 300 lokacija po klasi. Ukupan broj slika u skupu podataka je $108 * 300 * 5 = 162.000$.

4. Pretprocesiranje podataka

Pre upotrebe slika za treniranje modela, rezolucija svih slika će biti prepolovljena kako bi se smanjilo vreme treniranja i izbegao problem overfit-ovanja. Rezolucija pojedinačne slike će biti 320×230 piksela.

Kako bi povećali skup podataka, izvršićemo transformaciju slika korišćenjem vertikalne refleksije (obrtanjem leve i desne strane slike). Na ovaj način se kreira augmentovani skup podataka duplo veći od originalnog^[1].

Za rešavanje problema ovog projekta poredilo bi se nekoliko metoda, od kojih bi neke kao ulaz zahtevale da 5 slika budu spojene u jednu panoramu.



Figura 4: Kreiranje panorame na osnovu pet slika jedne lokacije

5. Metodologija

Za klasifikaciju slika koristiće se konvolucijska neuronska mreža (*eng. CNN*). *CNN*^[3] je naročito pogodan za ovakve vrste problema. Bazira se na upotrebi *kernels* pomoću kojih se konvolucijom kreiraju *feature maps*. Nakon nekoliko slojeva konvolucije rezultujući set *feature maps* se obrađuje pomoću potpuno povezanih slojeva (*eng. fully connected layers*), čiji rezultat predstavlja vektor dimenzije N , gde N predstavlja ukupan broj klasa. Ovaj vektor se zatim transformiše korišćenjem softmax funkcije^[4], čime se dobija distribucija verovatnoće pripadnosti ulazne slike svakoj klasi.

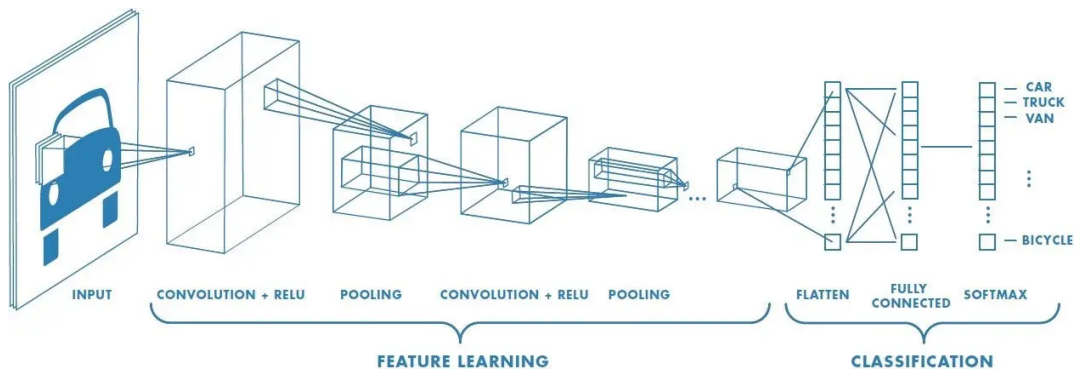


Figura 5: Ilustracija konvolucijske neuronske mreže

ResNet^[5] predstavlja unapređenje osnovnog CNN modela uvođenjem *ResNet blokova*. ResNet blokovi su u osnovi obične konvolucijske mreže, međutim ono što ih čini posebnim jeste način na koji se nadovezuju jedan na drugi. Kao ulaz jednog ResNet bloka se ne koristi samo izlaz, već takođe i ulaz prethodnog bloka. Na ovaj način ResNet arhitektura smanjuje problem nestajanja gradijenta (*eng. vanishing gradient problem*), čime omogućava uspješnije treniranje složenijih modela. *Inception*^[6] model predstavlja još jednu od nadogradnji klasičnog CNN modela. Ideja inception modela se bazira na korišćenju *kernels* različitih dimenzija unutar istog mrežnog sloja.

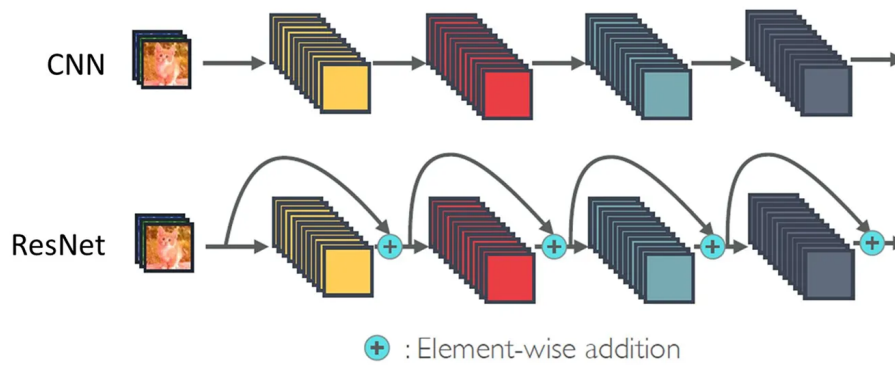


Figura 6: Ilustracija ResNet neuronske mreže

Transfer learning je metoda pomoću koje se deo ili svi parametri već istreniranog modela koriste kao parametri novog modela. Novi model može biti namenjen rešavanju drugačijeg problema, gde se za njegovo treniranje koristi drugačiji set podataka. Istrenirani model koji bi služio za potrebe ovog projekta je Inception ResNet model istreniran nad [ImageNet](#) skupom podataka. *ImageNet* se sastoji od 10.000.000+ slika i 10.000+ kategorija objekata.

U projektu bi se poredile različite metode integracije slika sa iste lokacije^[2]. Eksperimentisalo bi se sa poređenjem ranog i srednjeg nivoa integracije.

- Kod rane integracije, slike se spajaju u jednu [panoramu](#) koja predstavlja ulaz samog modela. Na ovaj način su podaci agregirani pre ulaska u model.
- Kod srednje integracije, slike ne spajamo u panoramu, već treniramo model da prima pojedinačne slike kao ulaz. Ovako istreniranom modelu uklanjamo potpuno povezane slojeve, ostavljajući model koji kao izlaz ima set feature mapa. Ovakvom modelu zasebno prosleđujemo svaku od slika lokacije dobijajući grupe feature mapa za svaku od ulaznih slika. Ove grupe feature mapa možemo objediniti upotrebom *Deep Set*^[7] kako bismo dobili krajnji rezultat klasifikacije.

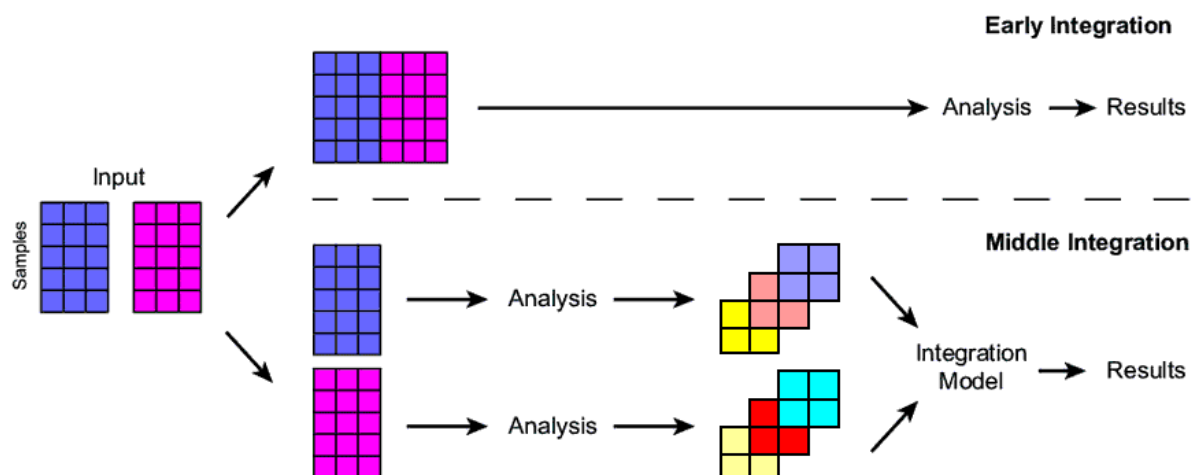


Figura 7: Ilustracija različitih nivoa integracija

6. Evaluacija

Najjednostavniji način za dobijanje koordinata kao rezultata, jeste posmatrati problem kao regresiju, međutim regresija može biti ometena nepravilnim oblikom mape. Deljenje mape na ćelije i posmatranje problema kao problem klasifikacije zanemaruje oblik mape, ali ograničava preciznost na veličinu ćelije. Iz ovih razloga problem se posmatra kao kombinacija problema klasifikacije i regresije.

Klasifikacija se dobija korišćenjem potpuno povezanog sloja (*eng. fully connected layer*) koji kao ulaz prima *feature map-e* ResNetInceptionV2 modela. Ulaz za regresiju je skup rezultata klasifikacije i *feature map-a*.

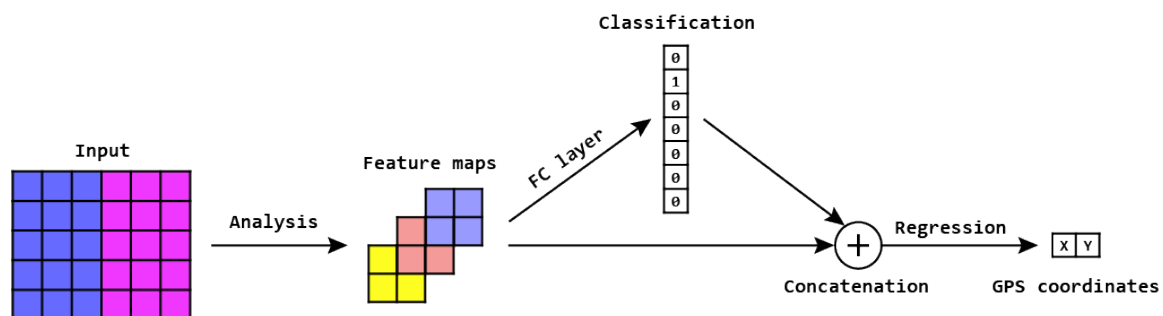


Figura 8: Prikaz procesa dobijanja rezultata klasifikacije i regresije

Loss vrednost rezultata klasifikacije se računa kao *Cross-Entropy Loss*^[8]. *Loss* vrednost rezultata regresije se računa kao srednja absolutna greška (*eng. Mean Absolute Error*), dok se za metriku koristi distanca izačunata *Haversine*^[9] metodom.

7. Tehnologije

Tehnologije koje ćemo koristiti u razvoju aplikacije i modela:

- **Python** programski jezik
- **TensorFlow** biblioteka za razvijanje modela
- **Geopandas** biblioteka za rad sa mapama
- **Matplotlib** biblioteka za vizuelizaciju podataka
- [Google Street View Static API](#) za prikupljanje skupa podataka

8. Literatura

- [1] Saleh Alwer,
GeoGuessr-Inspired Exploration of CNNs: Predicting Street View Image Locations,
[Istraživački rad](#), [GitHub](#)
- [2] Sudharshan Suresh, Nathaniel Chodosh, Montiel Abello,
DeepGeo: Photo Localization with Deep Neural Network,
[Istraživački rad](#)
- [3] Keiron O'Shea, Ryan Nash,
An Introduction to Convolutional Neural Networks,
[Istraživački rad](#)
- [4] GeeksforGeeks,
Softmax Activation Function in Neural Networks,
[Članak](#)
- [5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun,
Deep Residual Learning for Image Recognition,
[Istraživački rad](#)
- [6] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, Andrew Rabinovich,
Going Deeper with Convolutions,
[Istraživački rad](#)
- [7] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabás Póczos, Ruslan Salakhutdinov, Alexander J Smola,
Deep sets,
[Istraživački rad](#)
- [8] Kurtis Pykes,
Cross-Entropy Loss Function in Machine Learning: Enhancing Model Accuracy,
[Članak](#)
- [9] Yves Mercadier,
Haversine Distance,
[Članak](#)